C++ Programming Problem Set

Module: Arrays (1D, 2D, 3D, Pointer Math & Modern STL Containers)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Arrays & Contiguous Memory Layout

Integrates: I/O, Loops, Functions, Pointers, References & STL

Instructions & Pedagogical Guidelines

This problem set rigorously tests your understanding of C++ array architecture. You will write code manipulating 1D, 2D, and 3D contiguous memory layouts, utilize pointer arithmetic offsets, implement pass-by-address and reference parameter functions, and harness modern C++ STL components (`std::array` and `std::vector`). Ensure all array indices remain safely within valid bounds.

Level 1: 1D Arrays, Contiguous Storage & Pointer Math

Problem 1: Address Inspector & Pointer Arithmetic Verifier

1D Array | Pointer Decay | Base Address Math

Write a C++ program that reads 5 integers into a raw array `arr[5]`. Using a loop, print each element's value, its subscript access `arr[i]`, and its memory address obtained via both `&arr[i]` and pointer arithmetic `(arr + i)`. Calculate and display the exact byte difference between adjacent elements to verify contiguous memory allocation based on `sizeof(int)`.

Base Address Pointer Decay | Pointer Arithmetic | Contiguous RAM

Sample Input	Sample Memory Analysis Output
`Array: [10, 20, 30, 40, 50]`	`Element 0: Val=10 | Addr=0x1000 | *(arr+0)=10` `Element 1: Val=20 | Addr=0x1004 | Offset: +4 Bytes`

Problem 2: In-Place Frequency Counter & Partial Initializer

1D Array | Control Structures | Zero Initialization

Prompt the user to enter N scores (where `N <= 20`) ranging strictly between 0 and 5. Initialize a frequency tracking array `int freq[6] = {0};` relying on partial zero-initialization rules. In a single loop, populate the scores and update frequency counts inside `freq`. Print a frequency histogram table using nested loops.

Partial Array Zero-Filling | Direct Subscript Indexing | Nested Loop Visualization

Sample Input Scores	Sample Histogram Output

`Scores: 5, 2, 2, 0, 5, 2`	`Score 0: * (1)`
	`Score 2: *** (3)`
	`Score 5: ** (2)`

Problem 3: Dynamic Array Rotation via Modulo Offset

1D Array | Mathematical Indexing | Modular Functions

Create a function `void rotateArrayRight(int arr[], int size, int k)` that rotates the array to the right by k positions in-place or using an auxiliary buffer. Integrate logic to normalize `k = k % size` to handle rotation counts larger than array size. Test the function using user inputs in `main()`.

Array Decay to Pointer | Index Modulo Arithmetic | Pass-by-Address

Sample Input	Sample Output
`Array: [1, 2, 3, 4, 5], k = 2`	`Rotated Array: [4, 5, 1, 2, 3]`

Level 2: Multidimensional Arrays (2D Matrices & 3D Space)

Problem 4: Matrix Transposition & Symmetry Checker

2D Array | Row-Major Mapping | Matrices

Write a program that takes a square 2D matrix `int mat[3][3]`. Implement two modular functions:
1. `void transposeMatrix(const int src[3][3], int dest[3][3])` to compute the matrix transpose.
2. `bool isSymmetric(const int mat[3][3])` that checks if `mat[i][j] == mat[j][i]` across all elements.
Output both the transposed grid and the symmetry evaluation boolean result.

```
// Matrix Transpose Signature (Column dimension mandatory in C++ function headers!): void transposeMatrix(const int src[3][3], int dest[3][3]); bool isSymmetric(const int mat[3][3]);
```

Sample Input Grid	Transposed Grid & Symmetry Result
`[[1, 2, 3], [2, 4, 5], [3, 5, 6]]`	`Transposed: [[1, 2, 3], [2, 4, 5], [3, 5, 6]]` `Status: SYMMETRIC MATRIX (TRUE)`

Problem 5: Row-Major Memory Offset Calculator

2D Array | Linear Address Math | Row-Major Order

Although a 2D matrix is conceptualized as a grid, physical RAM stores it in linear Row-Major order: `Address(A[i][j]) = Base + (i * Cols + j) * sizeof(Type)`. Declare a `2 x 3` integer array. Read values into the matrix, then cast the array base address to a single `int* ptr` pointer. Iterate through the array sequentially from `ptr[0]` to `ptr[5]` to prove Row-Major memory continuity.

Row-Major Physical Layout | Flattening 2D to 1D | Pointer Offset Access

2D Grid Input	1D Flattened Pointer Sequence
`Row 0: [5, 8, 9]` `Row 1: [10, 12, 11]`	`Linear Traversal via ptr*: 5, 8, 9, 10, 12, 11` `Base Addr + 0x00 to Base Addr + 0x14`

Problem 6: 3D Multi-Warehouse Inventory Tracker

3D Array | Triply-Nested Loops | Spatial Data

A company owns 2 warehouse branches, each having 3 storage floors, with 2 product aisles per floor. Represent this using a 3D array `int inventory[2][3][2]` (Pages x Rows x Columns). Write a program using triply-nested loops to populate stock counts, compute total stock across all locations, and identify the specific warehouse page, floor row, and aisle column holding the maximum inventory.

3D Array Navigation | Triply-Nested Loops | Spatial Data Processing

Sample Inventory Data	Sample Output Summary
`Page 0: {{4,5},{6,7},{8,0}}` `Page 1: {{11,12},{13,0},{2,1}}`	`Total Stock: 69 Units` `Max Stock: 13 Units at [Page 1][Floor 1][Aisle 0]`

Level 3: Modern C++ STL Containers (std::array & std::vector)

Problem 7: Bounds-Safe Modern Array Analyzer (std::array)

std::array | Safe .at() Access | Modern C++

Raw arrays lack runtime bounds checking and decay into pointers when passed to functions. Write a modern C++ program utilizing `std::array<int, 6>`. Implement a function `void processModernArray(const std::array<int, 6> &arr)` that uses range-based `for` loops and `.size()`. In `main()`, demonstrate safe element retrieval via `.at(index)` wrapped in a `try-catch (const std::out_of_range &e)` block.

```
// Modern Stack Array with Safe Access: std::array<int, 6> scores = {88, 92, 79, 95, 61, 100}; try { std::cout << scores.at(10); } // Throws std::out_of_range exception! catch (const std::out_of_range& e) { std::cout << "Out of bounds caught!"; }
```

Access Method	Safety Analysis
`rawArr[10]`	`Undefined Behavior / Memory Corruption (No Bounds Check)`
`stdArr.at(10)`	`Throws std::out_of_range Exception (Bounds Checked)`

Problem 8: Dynamic Heap Buffer & Resizing Engine (std::vector)

std::vector | Dynamic Resizing | Range-Based For

Unlike raw arrays, `std::vector` stores elements on the dynamic Heap and resizes automatically. Write a program that reads integers from a user until -1 is entered. Push items into a `std::vector<int>`, displaying its `.size()` and `.capacity()` after each insertion to observe dynamic memory reallocation. Write a helper function accepting `std::vector<int> &vec` to remove all odd numbers using `.erase()`.

Heap Buffer Resizing | Size vs Capacity | Vector Mutation Functions

Inputs Pushed	Growth Output Log
`Push: 10, 20, 30`	`Size: 3 | Capacity: 4 | Heap Allocation Auto-Expanded` `After Odd Erasure: [10, 20, 30]`

Problem 9: Cumulative Capstone: Image Pixel Matrix Filter

Comprehensive Integration | 2D Vector | Image Processing

Build a simplified grayscale image processor. Represent an `R x C` image using a 2D dynamic vector `std::vector<std::vector<int>> image` where pixel values range from 0 to 255. Implement functions:
1. `void applyThreshold(std::vector<std::vector<int>> &img, int threshold)` — Binarize pixels (>= threshold -> 255, else 0).
2. `void computeHistogram(const std::vector<std::vector<int>> &img, int &blackCount, int &whiteCount)` — Count total black (0) vs white (255) pixels using reference parameters.
Process a 4x4 matrix and output the original, filtered matrix, and pixel distribution report.

2D Dynamic Vector | Const Reference Parameters | Pixel Image Matrix Processing | Complete Module Capstone

Original 4x4 Pixels	Threshold (128) Applied	Summary Output
`[[45, 180], [210, 90]]`	`[[0, 255], [255, 0]]`	`Black Pixels (0): 2 | White Pixels (255): 2`